Kilambi Research Working Paper No. KR-WP-2026-29

Coordination Costs and Break-Even Conditions for Peer-to-Peer Inference of Open-Weight Models

Kilambi Research

October 2026

Abstract. This paper asks whether an opt-in network of consumer devices can serve open-weight language models at interactive speed and at a cost competitive with centralized serving. Five coordination designs are compared in a single accounting framework: centralized serving (A0), whole-request routing to full replicas on peers (A1), layer sharding across peers (A2), speculative decoding with a local draft and a sharded verifier (A3), and sparse mixture-of-experts serving in which the model itself is dispersed across devices, in both its standard per-layer form (A4N) and a restructured form with locality routing, expert caching, and early exit (A4R). Closed-form latency and cost expressions are derived for each design and tested in a discrete-event simulator that first reproduces the closed forms exactly in noiseless runs. Sharding never beats a smaller model run on the requester's own device at any tested wide-area round-trip time. Speculative decoding raises mean throughput but converts steady latency into bursty latency, and its break-even acceptance rate falls as round-trip time rises, from 0.42 at a 10 ms round trip to 0.17 at 150 ms. Replica routing on peers undercuts centralized cost only when centralized utilization is below a measured crossover of 0.28. Per-layer expert routing fails by two to three orders of magnitude on wide-area links. The restructured sparse design meets a 50 ms interactive target only within a single city and only at low sequential depth. When the fleet grows at fixed depth, the binding scaling term is the decay of cache coverage, not queueing. All quantitative results are (Modelled, C); no hardware was measured.

Keywords: distributed inference; peer-to-peer computing; mixture of experts; speculative decoding; cost modelling

Contents

1 Introduction **4**

2 Prior art and related work **5**

2.1 Collaborative inference systems 5

2.2 Decentralized training 5

2.3 Speculative decoding, and distributing it 6

2.4 Mixture-of-experts serving and the all-to-all wall 6

2.5 Verification of remote computation 7

2.6 Incentive networks and the accounting gap 7

3 The coordination model **7**

3.1 Setup and canonical parameters 7

3.2 The decode step 8

3.3 A0, A1, and A2: whole models in one place, or sharded 9

3.4 A3: speculative decoding over a sharded verifier 9

3.5 A4: the dispersed sparse model 10

3.6 Fleet scaling: proposition T-SCALE 11

3.7 Cost per committed token 12

3.8 Training bound and hypotheses 12

4 Simulator and validation **13**

4.1 Design 13

4.2 Validation 13

5 Results **14**

5.1 Headline latency 14

5.2 Hypothesis verdicts 14

5.3 Fleet scaling: T-SCALE 15

5.4 Use-case classification 17

6 Discussion **17**

6.1 Break-even conditions, collected 17

6.2 Two gaps in the literature 19

6.3 What a testbed phase would need to show 20

7 Limitations **21**

8 Conclusion **21**

A Device-class thresholds **25**

B Full results **25**

C Falsifiability **26**

1 Introduction

Open-weight language models can be copied to any device that can store them. Serving them is a different problem. A data centre concentrates accelerators behind a fast fabric and charges for the capacity whether or not it is used. A peer-to-peer network has the opposite profile: a large pool of consumer devices, each slow by data-centre standards, each connected by a wide-area link with a round-trip time of tens to hundreds of milliseconds, and each available only when its owner is not using it. The question this paper answers is narrow and quantitative: for which service classes, if any, can such a fleet serve open-weight models at interactive speed and at a cost that competes with centralized serving?

The paper compares five coordination designs under one accounting framework. A0 is centralized serving in a data centre. A1 routes each request whole to a peer that holds a full replica of the model. A2 shards one model across several peers and pipelines activations between them. A3 runs a small draft model on the requester's device and verifies blocks of draft tokens against a large model sharded across peers. A4 disperses a sparse mixture-of-experts model across the fleet, so that no device holds the whole model; it is evaluated in its standard per-layer form (A4N) and in a restructured form (A4R) that adds locality-first routing, a per-device expert cache, and early exit to cut the number of sequential remote steps per token.

Three results carry the paper. First, coordination that splits a single token's computation across wide-area links pays a round trip per split, and the payment is large relative to the compute it buys: sharding (A2) is slower than running a smaller model locally at every tested tier, and per-layer expert routing (A4N) misses an interactive latency target by two to three orders of magnitude. Second, the designs that keep each token's computation in one place, A1 and the restructured A4R at low depth, do close at some tiers: A1 meets a 50 ms P95 inter-token gap target at all three network tiers tested, and A4R meets it within a single city. Third, the economics invert the usual intuition about idle capacity. A peer fleet is cheaper than a data centre only when the data centre would run at low utilization; the measured crossover is a centralized utilization of 0.28 (Modelled, C).

The method is a closed-form model of per-token latency and of cost per committed token, checked by a discrete-event simulator that reproduces the closed forms exactly in noiseless runs (Section 4). Four hypotheses, fixed before the simulator was run, are tested in Section 5. One of them, H2, is reported as not sustained in its first stated form: an internal audit found an error in the equation the hypothesis was stated against, and the correction is printed in full in Section 3 rather than absorbed silently.

Section 2 reviews what the literature and the deployed networks establish, and where their evidence stops. Section 3 builds the coordination model. Section 6 collects the break-even conditions, states two gaps in the literature that this paper's measurements address, and specifies what a physical testbed phase would need to show. Section 7 lists the limitations, including the simplifications inside the simulator. All results from the model and the simulator are (Modelled, C): they are computed from stated parameter values, and no hardware was measured in this study.

2 Prior art and related work

2.1 Collaborative inference systems

Petals is the reference implementation of volunteer inference (Borzunov et al., 2023). It shards a dense model by layers across volunteer servers and pipelines activations between them, the design this paper calls A2. For BLOOM-176B it reports 1.71 sequential steps per second at round-trip times below 5 ms, falling to 1.23 steps per second at 100 ms, and batched throughput of 253.6 tokens per second at low round-trip time falling to 112.2 at 100 ms (Observed, B). The same affine decline in round-trip time is what the sharding model of Section 3 predicts; Section 4 uses the Petals numbers as a qualitative check on shape.

Parallax schedules models across a decentralized pool with a per-model choice between pipeline and tensor parallelism and a global scheduler (Tong et al., 2025). The authors report throughput up to 3.6 times and latency up to 3.2 times better than decentralized baselines, with averages of 1.58 and 1.66 times (Observed, B). Two consumer and prosumer systems occupy the local-network end of the design space. Exo runs tensor parallelism across home devices over LAN and Thunderbolt links (Exo Labs, 2026); its speedup figures are vendor claims with no independent wide-area benchmark. distributed-llama runs tensor parallelism across devices in C and publishes a measured synchronization payload of 32.9 MB per token for a 70B model spread over eight devices (Observed, B) (distributed-llama contributors, 2026); that payload is why tensor parallelism across peers is excluded from every arm in this paper. Prima.cpp runs 30B to 70B models on heterogeneous home clusters and reports a 70B model at 674 ms per token on four home devices over Wi-Fi (Li et al., 2026).

Physical wide-area tests are scarce. One three-machine test of pipeline inference across consumer internet links reached 1.615 tokens per second in its best configuration and 0.590 in its sealed configuration, with 95 to 98 per cent of each cycle spent waiting, and its author judged the setup not viable for chat (Observed, B) (mmprotest, 2026). Production decentralized inference, where it exists, avoids sharding: the serving networks examined in the literature surveys route each request whole to a node holding a full replica, the A1 design (Observed, B).

2.2 Decentralized training

Training tolerates latency that inference cannot, because a training step can amortize a synchronization over hundreds of local steps. SWARM trained a 1.01B-parameter model on 400 preemptible T4 GPUs at links below 400 Mb/s (Ryabinin et al., 2023) (Observed, B). DiLoCo runs about 500 inner steps between synchronizations (Douillard et al., 2023); OpenDiLoCo reproduced the method across two continents at 90 to 95 per cent compute utilization (Jaghouar et al., 2024) (Observed, B), although the OpenDiLoCo repository is no longer maintained. INTELLECT-1 trained a 10B model on one trillion tokens across at most 14 nodes on three continents, at 83 to 96 per cent utilization and 36 to 41 per cent model FLOPs utilization, with a global synchronization of about ten minutes amounting to 2 to 10 per cent of training time (Prime Intellect, 2024) (Observed, B). INTELLECT-2 broadcast 62 GB of model state in 14 minutes, an effective 590 Mb/s (Prime Intellect, 2025) (Observed, B). DisTrO claims a bandwidth reduction of up to 10,000 times; its documented test reduces 74.4 GB to 86.8 MB, a factor of about 857 (Observed, B) (Nous Research, 2024). Against this record, the follow-on INTELLECT-3 model, a 106B mixture-of-experts model, was trained on 512 co-located H200 accelerators rather than on internet compute, a retreat from

the decentralized design reported in secondary coverage (OBSERVED, C). Section 3 treats training only through hypothesis H4, which asks what synchronization interval the arithmetic requires; the simulator serves inference and does not test training.

2.3 Speculative decoding, and distributing it

Speculative decoding drafts several tokens with a small model and verifies them in one pass of the target model, committing a random number of tokens per verification cycle (Leviathan et al., 2023; Chen et al., 2023). Reported speedups on data-centre hardware are 2.0 to 3.5 times (Leviathan et al., 2023), 2.2 to 3.6 times for multi-head drafting (Cai et al., 2024), and 3.0 to 6.5 times with acceptance lengths up to 7.5 for the EAGLE-3 feature-based drafter (Li et al., 2024, 2025b), all (OBSERVED, B) and all measured where draft and target share a fast interconnect.

Two studies distribute the draft and the verifier across a network, and both carry a caveat that limits what they establish for wide-area serving. Song et al. built decentralized speculative decoding with a semantic-adaptive acceptance mechanism and report speedups of 2.56 times on HumanEval and 2.59 times on GSM8K (Song et al., 2025) (OBSERVED, B); their testbed is A800-80GB nodes on 100 Gb/s InfiniBand, so the result measures the algorithm on a data-centre fabric, not on consumer links. An edge-cloud study reports a crossover at about 50 to 60 ms round-trip time, below which distribution wins and above which cloud-only decoding wins (Yu et al., 2025) (OBSERVED, B); the simulator in Section 4 recovers a break-even structure of the same kind from first principles. Verification traffic is the second constraint: shipping a full vocabulary distribution per draft token collapses at 20 Mb/s links, while greedy token-ID verification does not, and wireless-informed semantic verification cuts latency by up to 31.4 per cent at 500 Mb/s (Liu et al., 2026) (OBSERVED, B).

2.4 Mixture-of-experts serving and the all-to-all wall

DeepSeek-V3 is the reference point for expert parallelism at scale: 256 routed experts and one shared expert per mixture-of-experts layer, eight experts active per token, at most four nodes touched per token (DeepSeek-AI, 2025b,a) (OBSERVED, B). Its communication library, DeepEP, reaches 153 to 158 GB/s within a node and 43 to 58 GB/s between nodes, the latter with eight 400G network cards per node (DeepSeek-AI, 2026a) (OBSERVED, B). Fine-grained experts lower loss at fixed active parameters (Krajewski et al., 2024), and production stacks replicate hot experts to balance load (DeepSeek-AI, 2026b; Dai et al., 2024). A secondary derivation from router statistics puts the effective communication-only ceiling of the DeepSeek-V3 routing pattern at about 67 tokens per second on a 400 Gb/s fabric (OBSERVED, B) (blokzdev, 2026); Section 3 derives the same pattern's volume directly and compares the two bounds.

Three techniques reduce the communication the pattern needs. Locality- first routing and speculative pre-scheduling raised the local activation rate by 43 per cent on DeepSeek-V2 and 61 per cent on Mixtral (Jiang et al., 2024), and cut single-layer latency by 40 to 69 per cent (Li et al., 2025a) (OBSERVED, B). Expert caching at the serving device reports up to 3.93 times throughput and about 72 to 75 per cent lower latency in data-centre and edge settings (Du et al., 2024; Chen et al., 2025) (OBSERVED, B). Depth reduction, by mixture-of-depths routing, reports sampling up to 50 per cent faster at matched quality (Raposo et al., 2024), and confident early exit reports up to three times less compute (Schuster et al., 2022), both (OBSERVED, B). No source in this pool

evaluates the three techniques jointly, on wide-area links, against an interactive latency target; that combination is the A4R design of Section 3 and one of the two literature gaps claimed in Section 6.

2.5 Verification of remote computation

A requester who pays a peer for tokens needs evidence the tokens came from the agreed model. TOPLOC stores a locality-sensitive proof of each inference and cuts proof storage by more than a factor of 1,000, to 258 bytes per 32 tokens against 262 KB for a naive record, and it detects model, prompt, and precision substitution (Ong et al., 2025) (OBSERVED, B). Zero-knowledge proof systems remain far from per-token use: reported systems prove a single 13B inference in under 15 minutes, and proof overheads on small models run at 100 to 1,000 times the inference cost (OBSERVED, B). Deployed verification therefore relies on refereed re-execution, as in Gensyn's Verde design, which needs at least one honest replica, or on trusted-execution attestation, which moves trust to the chip vendor. No production network cryptographically proves each token it sells (OBSERVED, B).

2.6 Incentive networks and the accounting gap

Token-incentivized compute networks report device counts, lease counts, and token burns. They seldom report delivered, verified compute, and no independent audit in the survey pool establishes a high utilization rate for any of them. Akash recorded 34,300 new leases in the fourth quarter of 2025, up 28 per cent quarter on quarter, with revenue of USD 463,200, down 46 per cent (Messari, 2026) (OBSERVED, B). Render burned 530,171 RENDER between January and September 2025, up 279 per cent year on year, with AI compute still a minority of its work (Messari, 2025) (OBSERVED, B). io.net's reported connected-device count fell from about 600,000 to about 10,000 after an authentication fix removed counterfeit devices (Cointelegraph, 2024) (OBSERVED, B). Analyst estimates put Bittensor's external revenue at USD 28 to 35 million annualized against token emissions above USD 300 million (OBSERVED, C). The gap matters for this paper's question: a supply count is not a utilization measurement, and the cost comparison in Section 3 turns on utilization, which is the quantity the networks do not publish.

3 The coordination model

3.1 Setup and canonical parameters

A requester holds a prompt and wants a stream of tokens. A token is *committed* when the serving design finalizes it; rejected draft tokens in A3 are never committed. Two latency quantities are tracked throughout: the inter-token gap, the time between successive committed tokens as the requester sees them, and time to first token (TTFT). The interactive target used for classification is a P95 inter-token gap of 50 ms, about 20 tokens per second, set as a design parameter (ASSUMED, C).

Four canonical models carry the analysis (Table 1). M8 and M70 are dense models of 8B and 70B parameters; D1 is a 1B draft model; L3 is a 3B model small enough to run on the requester's own device, and it is the local baseline every networked design must beat. Weights at b bytes per

Table 1. Canonical models, weights, and per-token payloads.

Model	Parameters	Weights INT4	Weights FP16	Boundary payload
M8	8B	4.0 GB	16 GB	8 KiB (hidden 4096)
M70	70B	35 GB	140 GB	16 KiB (hidden 8192)
D1 (draft)	1B	0.5 GB	2 GB	n.a.
L3 (local)	3B	1.5 GB	6 GB	n.a.

Notes: Key-value cache growth is 128 KiB per token for M8 and 320 KiB per token for M70 at FP16, so an 8k context holds 1.0 GiB (M8) or 2.5 GiB (M70). Boundary payload is one hidden vector per decode token. Source: model dimensions as published for the Llama-class reference models (RECONSTRUCTED, B); payload sizes derived from the dimensions (RECONSTRUCTED, B).

Table 2. Device classes and single-step decode time.

Class	BW (GB/s)	Memory (GB)	Rate (USD/hr)	Step (ms)
Data-centre GPU	3,350	80	2.00	1.7
Edge server	2,000	n.a.	0.50	2.9
Consumer dGPU	500	12	0.25	11.4
Laptop iGPU	100	n.a.	0.10	57
Smartphone SoC	60	n.a.	0.05	95
IoT / TV SoC	25	3	0.02	229

Notes: The data-centre class is H100-class hardware; the consumer dGPU is RTX 4070 class. Step time is $\tau = W/(\eta \cdot \mathrm{BW})$ with decode efficiency $\eta = 0.7$ (Eq. 1), computed for the M8 INT4 weight set of 4.0 GB. The edge server's M70 INT4 step is 25.0 ms. The sunk rate for a consumer dGPU is USD 0.06 per hour. Source: hardware specifications and prices (ASSUMED, C); step times derived (RECONSTRUCTED, C).

parameter occupy $W = P \cdot b$ bytes. One pipeline boundary crossing per decode token carries one hidden vector, and the key-value cache grows linearly with context (OBSERVED, B).

Six device classes span the fleet (Table 2). Network tiers are defined by median round-trip time: 10 ms within a city, 45 ms continental, 135 ms intercontinental, with a default access link of 100 Mb/s (ASSUMED, C). Prices are 2026 market rates as listed in Table 2; a sunk regime, in which the device is already owned and idle, prices a consumer GPU at USD 0.06 per hour against USD 0.25 at market rates (ASSUMED, C).

3.2 The decode step

Autoregressive decode at batch size one is memory-bound: each step reads the weights once. The step time on a device with memory bandwidth BW is

$$\tau(W, \mathrm{BW}) = \frac{W}{\eta \cdot \mathrm{BW}}, \qquad \eta = 0.7, \tag{1}$$

which gives 1.7 ms for M8 INT4 on a data-centre GPU and 11.4 ms on a consumer dGPU (RECONSTRUCTED, C). Batching amortizes the weight read across B concurrent sequences; throughput rises with B up to a saturation batch B_{sat} (64 in the canonical parameterization) and latency grows past it (ASSUMED, C). A remote step adds one network traversal,

$$h = \frac{\mathrm{RTT}}{2} + \frac{s}{B_{\mathrm{link}}}, \tag{2}$$

Table 3. Canonical A2 chain: M70 INT4, four stages on consumer dGPUs.

Link condition	RTT (ms)	Single-stream gap (ms)
LAN	1	109
City	10	132
Continental	45	219
Long continental	90	332
Intercontinental	135	444

Notes: Stage step 25.0 ms; gap from Eq. (3). The implied chain ceiling falls from 37 tokens per second on a LAN to 11 intercontinental at a batch of 16. Source: computed from Table 2 parameters (RECONSTRUCTED, C).

where s is the payload. At 8 to 16 KiB per boundary crossing and 100 Mb/s, the transfer term is 0.7 to 1.3 ms, small next to RTT/2 at continental and intercontinental tiers. Latency on wide-area links is bought in round trips, not in bytes, until payloads reach the megabyte scale of Section 3.5.

3.3 A0, A1, and A2: whole models in one place, or sharded

A0 serves from a data centre. Its single-stream gap is the step time of Eq. (1) plus one traversal to the requester. A1 places full replicas on peers and routes each request whole to one replica; its gap is the replica's step time plus queueing at the replica, bounded by provisioning replicas at a headroom utilization $\rho_{\max} = 0.7$ (ASSUMED, C). Neither design splits a token.

A2 splits the model. With N pipeline stages on homogeneous peers, the single-stream gap is

$$\text{gap}_{A2} = \sum_{i=1}^{N} \tau_i + (N+1) \cdot h. \tag{3}$$

Sharding divides memory, not latency: with homogeneous peers each stage still reads its shard once per token, so the compute sum is the full step time of the unsharded model on the same device class, and the traversals are pure addition. Heterogeneous chains are worse, because the slowest stage sets the pace and the terms add. Table 3 gives the canonical case, M70 INT4 in four stages on consumer dGPUs. Equation (3) is an upper-bound model: real pipelines overlap some transfers with compute, so the expression is correct in shape, affine in RTT with slope $(N+1)/2$, and conservative in level (RECONSTRUCTED, C). Tensor parallelism is excluded from all arms: its synchronization payload of 32.9 MB per token for M70 over eight devices (OBSERVED, B) gives a transfer time of 2.6 s per token at 100 Mb/s before any latency term.

3.4 A3: speculative decoding over a sharded verifier

A draft model on the requester's device proposes k tokens; one pass of the target model verifies the block. With per-token acceptance probability α, the expected committed tokens per cycle are the standard result (Leviathan et al., 2023)

$$E(\alpha, k) = \frac{1 - \alpha^{k+1}}{1 - \alpha}. \tag{4}$$

Table 4. A3 break-even against A2 ($k = 8$, phone draft, $t_d = 11.9$ ms).

RTT (ms)	L_v (ms)	α^*	Gap at $\alpha = 0.75$ (ms)	Speedup vs. A2
10	132	0.42	61	2.2x
45	219	0.30	85	2.6x
150	482	0.17	156	3.1x

Notes: α^* solves $\text{gap}_{A3} = \text{gap}_{A2}$ with the corrected Eq. (5); see the audit footnote attached to that equation for the first-pass values. Gaps are means over committed tokens; the P95 gap is close to the full cycle time (Section 5). Source: computed (RECONSTRUCTED, C).

Over a sharded verifier the cycle time is k local draft steps plus one verifier traversal L_v, the A2 traversal of Eq. (3) counted once, and the mean gap is

$$\text{gap}_{A3} = \frac{k \cdot t_d + L_v}{E(\alpha, k)}. \tag{5}$$

[1] Setting gap_{A3} equal to the A2 gap and solving for α gives the break-even acceptance rate α^* of Table 4. Two consequences follow. First, α^* falls as RTT rises: a slow verifier makes each cycle expensive, so fewer acceptances per cycle still pay. Second, the mean in Eq. (5) hides the burst structure of the output: tokens inside an accepted block are committed together, so the P95 gap is close to the full cycle time $k \cdot t_d + L_v$, not to the mean. The design constraint used in provisioning is $k \cdot t_d / L_v \leq 0.6$ (ASSUMED, C), which keeps draft time from dominating the cycle. A3 cannot beat an uncontended A1 replica of the same target model in single-stream latency at any α; its advantage appears under verifier contention, where one verification pass serves the drafts of many requesters. For the canonical edge verifier, A1 service saturates at 36 concurrent streams and the two designs reach gap equality near 140 streams per verifier (RECONSTRUCTED, C).

3.5 A4: the dispersed sparse model

A4 disperses a mixture-of-experts model across the fleet. The canonical sparse model has 256 experts of 100M parameters each (50 MB at INT4), eight active experts per token, and 25.6B parameters in total, 12.8 GB at INT4 (ASSUMED, C).

In the naive form A4N, each mixture-of-experts layer routes its token to experts on other devices, the DeepSeek-V3 pattern of 58 routed layers with two all-to-all exchanges per layer (DeepSeek-AI, 2025b). The per-token payload across all layers is

$$V = 3 \text{ bytes} \times \text{top-}k \times \text{hidden} \times \text{layers} = 9.98 \text{ MB} \approx 9.5 \text{ MiB}, \tag{6}$$

about 1,200 times the single boundary crossing of a dense model (RECONSTRUCTED, C). Latency alone, at a sequential depth of $D_{\text{naive}} = 58$ round trips, is 1.74 s per token at a 30 ms RTT and 10.4 s at 180 ms; the volume ceiling is 1.25 tokens per second at 100 Mb/s (RECONSTRUCTED, C).

[1] Audit trail. In the Phase P0 model document, the cycle time of Eq. (5) was first written with the round trip charged twice: the traversal L_v already begins and ends at the requester, and the first-pass form added a further round trip on top of it. The first-pass break-even acceptance rates were $\alpha^* = 0.45$, 0.39, and 0.34 at round-trip times of 10, 45, and 150 ms. A cross-check against the discrete-event simulator exposed the discrepancy, because the simulator charges one round trip per verification cycle. The corrected form printed here gives $\alpha^* = 0.42$, 0.30, and 0.17. Hypothesis H2 was stated against the first-pass numbers; its verdict in Section 5 is reported against both forms, and the distinction changes the verdict.

Table 5. Maximum sequential remote depth $D_{\max}$ by round-trip time.

	RTT (ms)					
Requester class	5	10	20	45	90	135
Consumer dGPU ($t_{\text{comp}} = 1.1$ ms)	8	4	2	1	0	0
Smartphone ($t_{\text{comp}} = 9.5$ ms)	7	3	1	0	0	0
IoT / TV SoC ($t_{\text{comp}} = 22.9$ ms)	4	2	1	0	0	0

Notes: Budget $L = 50$ ms. At a 100 ms budget the dGPU row reads 17, 9, 4, 2, 1, 0. A value of 0 means no remote step fits the budget at that tier. Source: computed from Eq. (8) parameters (RECONSTRUCTED, C).

The secondary derivation cited in Section 2 puts the effective ceiling at about 67 tokens per second on 400 Gb/s of fabric (OBSERVED, B). The two bounds differ by a factor of about 80 because the volume bound counts bytes and the effective bound counts dependent messages; on wide-area links the message count binds. Interactive expert parallelism of this form needs a fabric in the 10 GB/s class with sub-millisecond latency (RECONSTRUCTED, C), which consumer links are not.

The restructured form A4R cuts the sequential depth D, the number of remote steps on the critical path of one token, by three means: locality-first routing, which keeps a step on the current device when the router permits; a per-device hot cache of $C = 16$ experts (800 MB); and early exit at a fraction $e = 0.5$ of depth. With expert popularity following a Zipf law of exponent 1 over E experts, the cache covers a share of invocations

$$\varphi = \frac{H_C}{H_E}, \qquad H_n = \sum_{i=1}^{n} \frac{1}{i}, \tag{7}$$

which is $\varphi = 3.38/6.12 = 0.55$ at $C = 16$, $E = 256$ (RECONSTRUCTED, C). Remote steps per token are then binomial in D with miss probability $1 - \varphi$, and the P95 gap is

$$\text{gap}_{A4R} \approx t_{\text{comp}} + e \cdot D \cdot \text{RTT} + (1 - e) \cdot Q_{0.95}[\text{Binomial}(D, 1 - \varphi)] \cdot (\text{RTT} + s/B_{\text{link}}), \tag{8}$$

where $t_{\text{comp}} = a \cdot w_e/(\eta \cdot \text{BW}_{dev})$ for $a = 8$ active experts of $w_e = 50$ MB gives 1.1 ms on a dGPU, 5.7 ms on an iGPU, 9.5 ms on a phone, and 22.9 ms on an IoT SoC (RECONSTRUCTED, C). A latency budget L then caps the depth: $D_{\max}$ is the largest D with $D \cdot \text{RTT} + t_{\text{comp}} \le L$ (Table 5). The design criterion is that interactive A4R closes at a tier if and only if the operating depth after locality, caching, and early exit satisfies $D \le D_{\max}$ at that tier, with the tail checked by Eq. (8) rather than by the mean.

3.6 Fleet scaling: proposition T-SCALE

Replication is what lets a popular expert survive its own popularity. The top expert under Zipf-1 at $E = 256$ draws a share $p_1 = 0.163$ of invocations, and the top 26 experts draw 62 per cent (RECONSTRUCTED, C). Provisioning holds each copy at or below $\rho_{\max}$ by the rule $r_i \ge \lceil \lambda_i t_e / \rho_{\max} \rceil$ for expert i with arrival rate λ_i and service time t_e; at an aggregate 10,000 tokens per second the rule gives $r_1 \ge 3$, and replicating the top 26 experts fourfold costs a 31 per cent device premium over the unreplicated minimum (RECONSTRUCTED, C). Hedged requests, issuing a step to two holders and taking the first answer, cut the slow-step probability from q to about q^2 at up to twice the step cost (RECONSTRUCTED, C).

Proposition T-SCALE. At fixed $(D, \text{RTT tier}, t_{\text{comp}}, \rho_{\max})$, the per-token latency distribution is independent of fleet size F while total parameters and aggregate throughput grow in proportion to F. The proposition holds under eight conditions: replication funded in proportion to load; a cache covering a constant share of popularity mass; geographic routing discipline; discovery cost constant in F; a stationary device population; stationary churn; no central bottleneck; and constant external load per device. The second condition is the fragile one, and the test in Section 5 is run with its failure mode built in: the cache is held at 16 experts while the number of unique experts grows as $E_u = 4F$, so φ falls as F grows. The proposition is therefore tested in amended form: latency may move with F only through the φ term of Eq. (8), and any further F-dependence, in particular a queueing term, rejects the amendment as well.

3.7 Cost per committed token

All designs are charged by one identity. If a serving period produces N_c committed tokens, then

$$C_{\text{token}} = \frac{\sum_j r_j T_j + p_B B_{\text{net}} + p_m M}{N_c}, \tag{9}$$

where r_j is the rate of device class j, T_j its device-time (counting rejected drafts, duplicate hedged work, state transfer, and coordination as well as accepted service), B_{net} the bytes moved at price p_B = USD 0.01 per GB, and M the requester-side messages at p_m = USD 10^{-7}, prices (ASSUMED, C). Committed tokens only are counted: work spent on rejected drafts and lost hedges raises cost without raising N_c. For the centralized arm, cost per token is a function of the share of capacity in use, u:

$$C_{A0}(u) = \frac{r_{\text{DC}}}{3600 \cdot \Theta_{\max} \cdot u} = \frac{\text{USD } 0.0222}{u} \text{ per million tokens}, \tag{10}$$

at $\Theta_{\max}$ = 25,000 tokens per second for M8 FP16 (RECONSTRUCTED, C). A1 at market peer rates costs USD 0.064 per million tokens at full utilization of the replica fleet and USD 0.084 at the $\rho_{\max} = 0.7$ provisioning headroom (RECONSTRUCTED, C); the two cost curves cross where $C_{A0}(u) = C_{A1}$, at $u^* = 0.35$ at full utilization and $u^* \approx 0.26$ with headroom. In the sunk regime the A1 cost falls by about the rate ratio $0.06/0.25 = 0.24$, under the A0 curve at every u in $(0, 1]$, so the sunk regime has no interior crossover bound.

3.8 Training bound and hypotheses

Training enters through one bound. A synchronous data-parallel step for M8 at FP16 moves $G = 16$ GB of gradients; at a 50 Mb/s uplink the synchronization takes 5,120 s against a step time of about 2 s, and keeping synchronization at 10 per cent of step time needs an uplink of about 1.3 Tb/s (RECONSTRUCTED, C). Low-communication methods run H local steps per synchronization, and feasibility at overhead share ε requires

$$H \;\geq\; \frac{G \cdot 8}{B_{\text{up}} \cdot \varepsilon \cdot t_{\text{step}}}, \tag{11}$$

which gives $H \geq$ 16,000 at 20 Mb/s, 6,400 at 50 Mb/s, and 542 at 590 Mb/s (RECONSTRUCTED, C).

The hypotheses fixed in the research design, before the simulator was run, are:

H1. Sharding (A2) beats a smaller local model on single-stream latency in no configuration with homogeneous peers at wide-area RTT. A single counterexample configuration kills it.

H2. The A3 break-even α^* lies in $[0.34, 0.45]$ at all tiers and moves by at most 0.15 across tiers, as stated against the first-pass form of Eq. (5).

H3. Under market peer pricing, batch cost curves for A0 and A1 cross at $u^* = 0.35 \pm 0.10$.

H4. Internet training of an 8B-class model is feasible only in the low-communication regime of Eq. (11); the observed operating points of deployed runs sit at or above the bound.

4 Simulator and validation

4.1 Design

The closed forms of Section 3 are mean-value statements. The quantities that decide the use-case classification are tail quantities, P95 gaps and P95 time to first token, under jitter, contention, and popularity skew, so the results are produced by a discrete-event simulator. The kernel runs generator-based processes on an event heap. A serving node is a slot model of Eqs. (1) and the batching rule of Section 3: solo service time is τ, service accelerates with concurrent batch up to $B_{\text{sat}} = 64$, and work in a slot is shared in the manner of generalized processor sharing. Latency runs are per stream, with prompts of 256 tokens and generations of 256 tokens. Batch runs for A2, A3, and A4 are per-token discrete-event simulations. Batch runs for A0 and A1, whose token rates reach about 22,500 per second, use a fluid model of the same slot equations, integrated in rate steps; this asymmetry is listed as a limitation in Section 7.

Round-trip time is drawn lognormally per tier with medians 10, 45, and 135 ms and $\sigma = 0.3$ on the log scale (ASSUMED, C). Packet loss probability is held at zero in the reported grid; the kernel implements retransmission, exercised in code, and the grid holds loss at zero to isolate the round-trip terms. Churn is off except in marked runs; when a peer departs, an operation in flight on it is not interrupted, and the failure is observed at the next operation. All runs use fixed seeds, and rerunning the run matrix reproduces the results file exactly. The simulator is the only instrument in this study: its outputs are (MODELLED, C).

4.2 Validation

Validation has three parts. First, in noiseless runs (fixed 45 ms RTT, no jitter, no loss, no churn, one stream) the simulator must reproduce the closed forms of Section 3 for each arm. Table 6 reports all twelve checks; every relative error is 0.00 per cent at the printed precision. Second, the expected committed tokens per A3 cycle at $\alpha = 0.7$ are compared against Eq. (4): simulator 3.228, closed form 3.199, a difference of 0.90 per cent, inside the 2 per cent tolerance. Third, a qualitative check against the Petals measurements of Section 2: the simulator's ratio of sequential rate at 5 ms RTT to that at 100 ms is 2.995 against the reported 1.39. The levels differ by design, because the simulated chain has four stages of a smaller model against Petals' many-stage 176B chain; the checked quantities are the direction of the decline and its affine shape, and the simulated gap slope of 2.5 ms per ms of RTT equals $(N + 1)/2$ for $N = 4$ exactly, as Eq. (3) requires. The third check is graded qualitative (MODELLED, C).

Table 6. Noiseless reproduction of the closed forms (fixed 45 ms RTT).

Check	Quantity	Simulator	Closed form	Rel. error	Result
V1-A0	gap (ms)	8.69	8.69	0.00%	Pass
V1-A1	gap (ms)	8.69	8.69	0.00%	Pass
V1-A2	gap (ms)	212.52	212.52	0.00%	Pass
V1-A3	cycle (ms), $\alpha = 1$	143.1	143.1	0.00%	Pass
V1-A4R	P95 gap (ms), $D = 1$	52.50	52.50	0.00%	Pass
V1-A4R	P95 gap (ms), $D = 4$	145.49	145.49	0.00%	Pass
V1-A4N	gap (ms), $D = 58$	3,757.51	3,757.51	0.00%	Pass
V1-A4N	committed tok/s	0.266	0.266	0.00%	Pass
V1-TS	φ, $F = 50$, $D = 2$	0.575	0.575	0.00%	Pass
V1-TS	φ, $F = 5{,}000$, $D = 4$	0.323	0.323	0.00%	Pass
V2	Committed per block, $\alpha = 0.7$	3.228	3.199	0.90%	Pass
V3	Petals rate ratio, 5 vs. 100 ms	2.995	1.39	n.a.	Qualitative

Notes: V1 and V2 checks are exact or tolerance checks against Section 3. V3 compares direction and affine shape only, for the reason given in the text; no relative error is defined for it. Source: simulator validation runs (MODELLED, C); Petals figure (OBSERVED, B) (Borzunov et al., 2023).

5 Results

5.1 Headline latency

Table 7 and Figure 1 report the P95 inter-token gap for each arm at the three network tiers, single stream, against the local baseline L3, a 3B INT4 model on the requester's own consumer dGPU with a P95 gap of 4.3 ms (MODELLED, C). No design that leaves the device beats the local model in single-stream latency, at any tier. A0 and A1 stay below 20 ms at every tier. A2 is slower than the local baseline by a factor of 32 to 126. A4N is slower by three orders of magnitude at the intercontinental tier. A4R crosses the 50 ms target only at the city tier, at depths up to $D = 4$ (Figure 2).

A3 shows the widest gap between its mean and its tail. At the city tier its mean committed rate is 16.9 tokens per second against 7.5 for A2, the 2.2 times speedup of Table 4, and the ratio grows with tier (2.8 continental, 3.2 intercontinental) (MODELLED, C). Its P95 gap, 230 ms at the city tier, sits at the full verification cycle time of 227 ms from Eq. (5), because tokens in an accepted block are committed together. Speculation over a sharded verifier buys throughput and verifier capacity; it does not buy streaming smoothness, and under a P95 target it fails at every tier (Table 9).

5.2 Hypothesis verdicts

H1: held. Analytically, Eq. (3) adds traversals to a compute sum that sharding cannot reduce with homogeneous peers. In the simulator, A2's P95 gap is 138, 254, and 543 ms at the city, continental, and intercontinental tiers against the local baseline's 4.3 ms (MODELLED, C). No tested configuration is a counterexample.

H2: not sustained, as first stated. The hypothesis put α^* inside $[0.34, 0.45]$ at all tiers with movement of at most 0.15 across tiers. The simulator measures $\alpha^* = 0.40$ at the city tier and a value at or below 0.20 at the intercontinental tier, where 0.20 is the floor of the acceptance grid, so

Table 7. P95 inter-token gap by arm and network tier (ms).

Arm	City	Continental	Intercontinental
L3 local baseline (3B, own dGPU)	4.3	4.3	4.3
A0 centralized	7	9	11
A1 replica routing	12	13	17
A2 sharding ($N = 4$)	138	254	543
A3 speculative ($k = 8$)	230	331	582
A4R restructured, $D = 1$	15	66	202
A4R restructured, $D = 2$	25	105	317
A4R restructured, $D = 4$	40	171	523
A4R restructured, $D = 8$	69	292	881
A4N per-layer ($D = 58$)	1,445	3,754	9,487

Notes: Single stream, 256-token generations. The L3 row does not depend on the tier because the model runs on the requester's device. A3 is measured over a sharded verifier with a phone-class draft at $\alpha = 0.75$. Source: simulator, matrix runs (MODELLED, C).

the true value sits at or below it. The movement is -0.20, outside the stated tolerance, and the interval claim fails at the intercontinental tier. The failure traces to the equation the hypothesis was stated against: the first-pass form of Eq. (5) charged the round trip twice (audit footnote, Section 3.4). The corrected closed form gives $\alpha^* = 0.42$ at the city tier and 0.17 at a 150 ms round trip, and the simulator agrees with the corrected form. The qualitative claim underneath H2 held in strengthened form: the break-even acceptance rate falls as round-trip time rises, by more than the first statement allowed. Restated against the corrected equation with a movement tolerance of 0.30, the measurements are consistent with the hypothesis.

H3: held. The A1 batch cost is USD 0.084 per million committed tokens, roughly flat in the offered centralized utilization. The A0 cost falls from USD 0.224 per million at $u = 0.1$ to USD 0.026 at $u = 0.9$. The curves cross at $u = 0.28$ (Figure 3), inside the predicted 0.35 ± 0.10 and between the two analytical values of Section 3, 0.35 at full utilization and about 0.26 at the provisioning headroom. In the sunk regime the A1 cost falls by about the rate ratio 0.06/0.25, and no interior crossover bound exists, as the model states.

H4: analytical only. The simulator serves inference and does not test training, so H4 stands or falls on the consistency of Eq. (11) with observed runs. The bound requires $H \geq 542$ inner steps at 590 Mb/s. DiLoCo operates at $H \approx 500$ (Douillard et al., 2023) and INTELLECT-1 at $H = 100$ with much larger steps (Prime Intellect, 2024), both (OBSERVED, B). The observed points sit at or near the bound, and no observed run contradicts it.

5.3 Fleet scaling: T-SCALE

The T-SCALE test grows the fleet from 50 to 5,000 devices at fixed depth, continental tier, replication $r = 3$, with unique experts $E_u = 4F$ and the per-device cache held at 16 experts. Total parameters grow from 20B to 2T. P95 gaps move from 107 to 121 ms at $D = 2$ and from 170 to 213 ms at $D = 4$ (MODELLED, C) (Figure 4). Replication on its own barely moves latency: across $r = 1, 3, 5$ the $D = 4$ P95 gap spans 166 to 170 ms at $F = 50$, 196 to 202 ms at $F = 500$, and 210 to 214 ms at $F = 5{,}000$. The whole F-dependence runs through cache coverage φ, which falls from 0.58 to 0.41 to 0.32 as the expert count grows against the fixed cache, as the amended proposition of Section 3

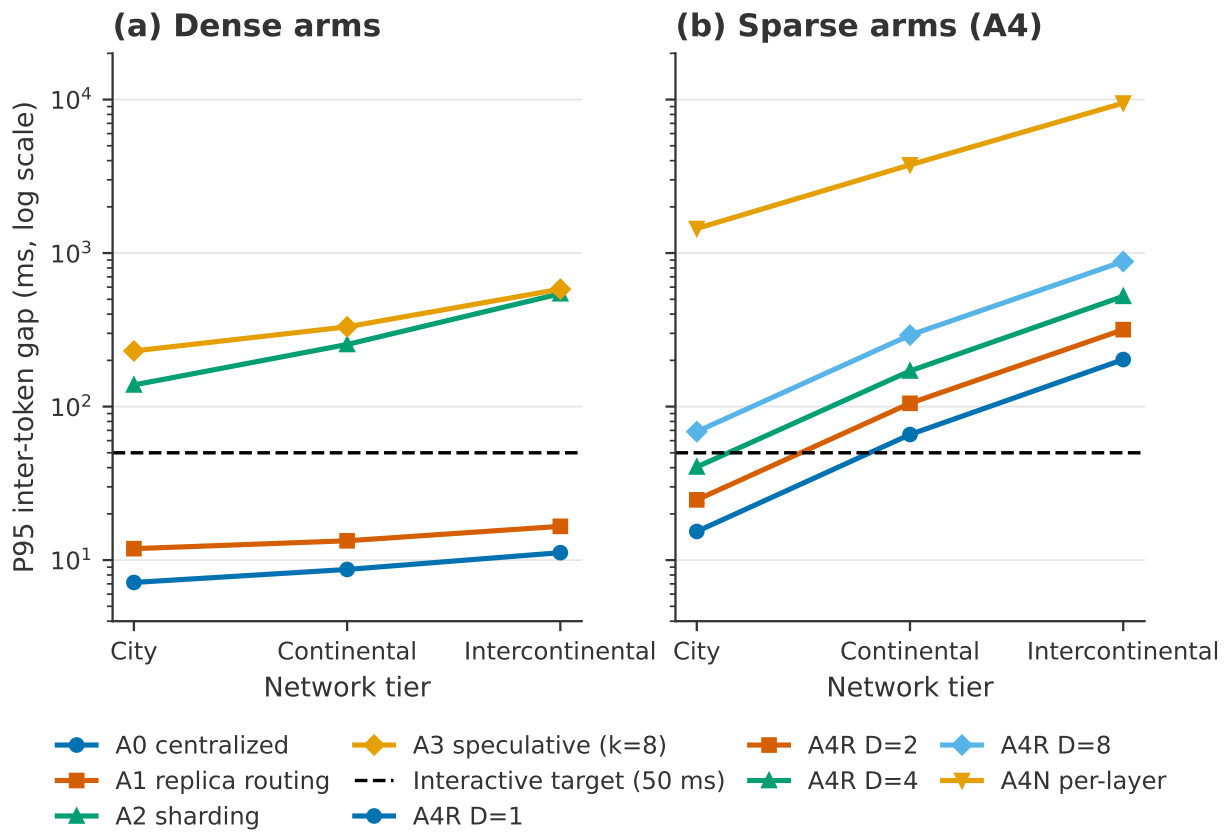


Figure 1. P95 inter-token gap by coordination arm and network tier, log scale. Panel (a): dense arms. Panel (b): sparse arms, A4R at four depths and A4N. The dashed line is the 50 ms interactive target. Source: simulator, matrix runs (MODELLED, C).

allows. Aggregate batch throughput grows in proportion to F, reaching 33,100 tokens per second at $F = 5{,}000$ and $D = 2$ (MODELLED, C).

The queueing clause of the proposition is a measured null. Queueing stretch, the ratio of batch step time to solo step time, is zero in every batch cell of the test. By Little's law the hottest device in the fleet sees an invocation concurrency of about 2 even at the largest tested aggregate rate of 17,900 tokens per second: the hottest expert draws about 13,600 invocations per second against a solo service time of 0.143 ms, an offered load near 1.9 Erlangs, spread across its replicas and far below the batching knee $B_{\text{sat}} = 64$. On these numbers the queueing clause would begin to bind only at about 30 times the largest tested aggregate rate (MODELLED, C).

Hedging is also a measured null at the point tested. With device speeds drawn lognormally ($\sigma = 0.5$) at $F = 500$, $r = 3$, hedging cut the share of critical paths running through the bottom speed decile, from 0.109 to 0.097 at $D = 2$ and from 0.233 to 0.195 at $D = 4$, and did not reduce the P95 gap: 118 to 115 ms at $D = 2$ and 196 to 203 ms at $D = 4$, with P99 unchanged at $D = 4$ (Table 8) (MODELLED, C). Most of the straggler penalty at this operating point sits inside one round trip of network jitter, which duplicating a step cannot remove, and the losing copy is charged at half service in the simulator's accounting.

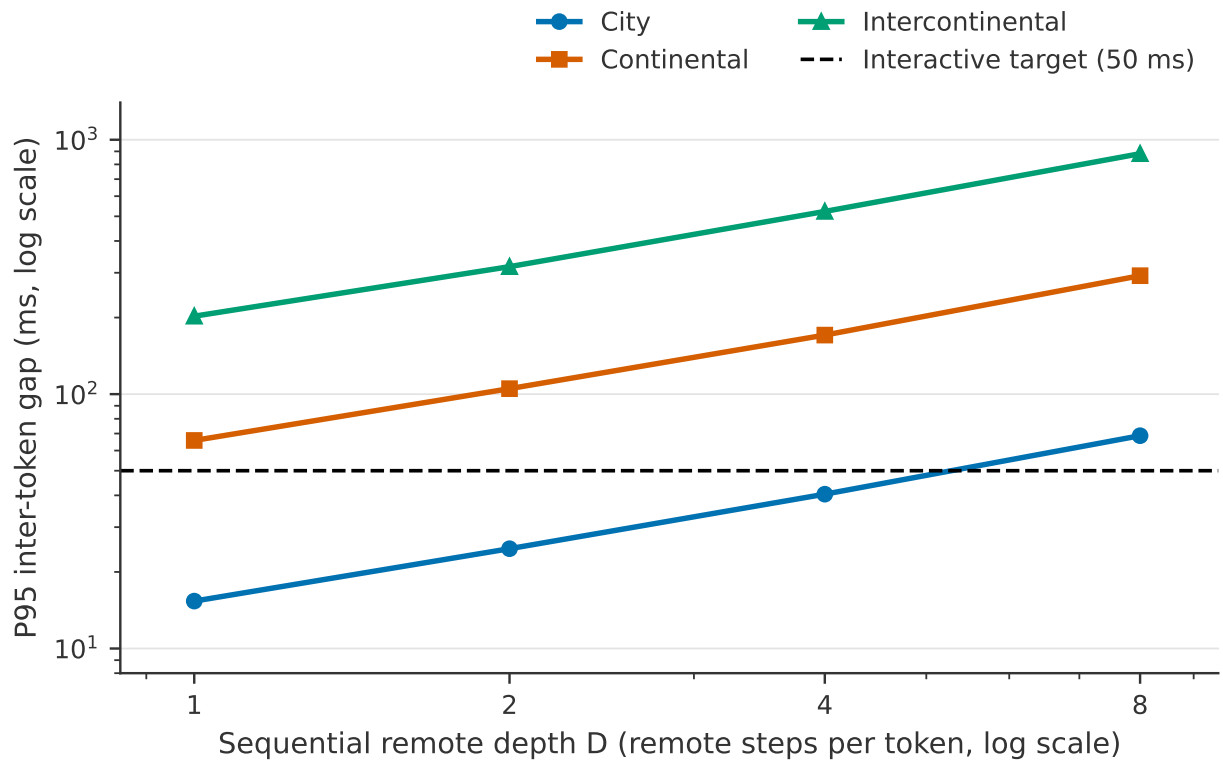


Figure 2. A4R P95 inter-token gap against sequential remote depth D, by network tier, log scales on both axes. The 50 ms target is met only at the city tier, for $D \leq 4$. Source: simulator, matrix runs (MODELLED, C).

5.4 Use-case classification

Table 9 classifies each design against three service targets: interactive chat (P95 gap at most 50 ms); an easy-query class of short generations (32 tokens, P95 TTFT at most 250 ms and P95 gap at most 50 ms, with early exit for A4R and $k = 4$ for A3); and batch work priced against the A0 reference at $u = 0.5$, USD 0.045 per million tokens at market rates. Chat passes for A0 and A1 at all tiers, and for A4R at the city tier only. The easy-query class passes for A0 at all tiers and for A1 at the city tier only; A1 fails it at the continental and intercontinental tiers on TTFT, not on the gap. Every sharded or dispersed design fails both interactive classes at every tier. On batch cost, no peer design beats the A0 reference at $u = 0.5$: A1 is closest at USD 0.084 per million, and A4N, at USD 112.184 per million, is three orders of magnitude away (MODELLED, C).

6 Discussion

6.1 Break-even conditions, collected

The results reduce to four break-even statements, each with the variable a deployment would have to control.

Replica routing (A1) is the only peer design that meets the interactive target at every tier, and its

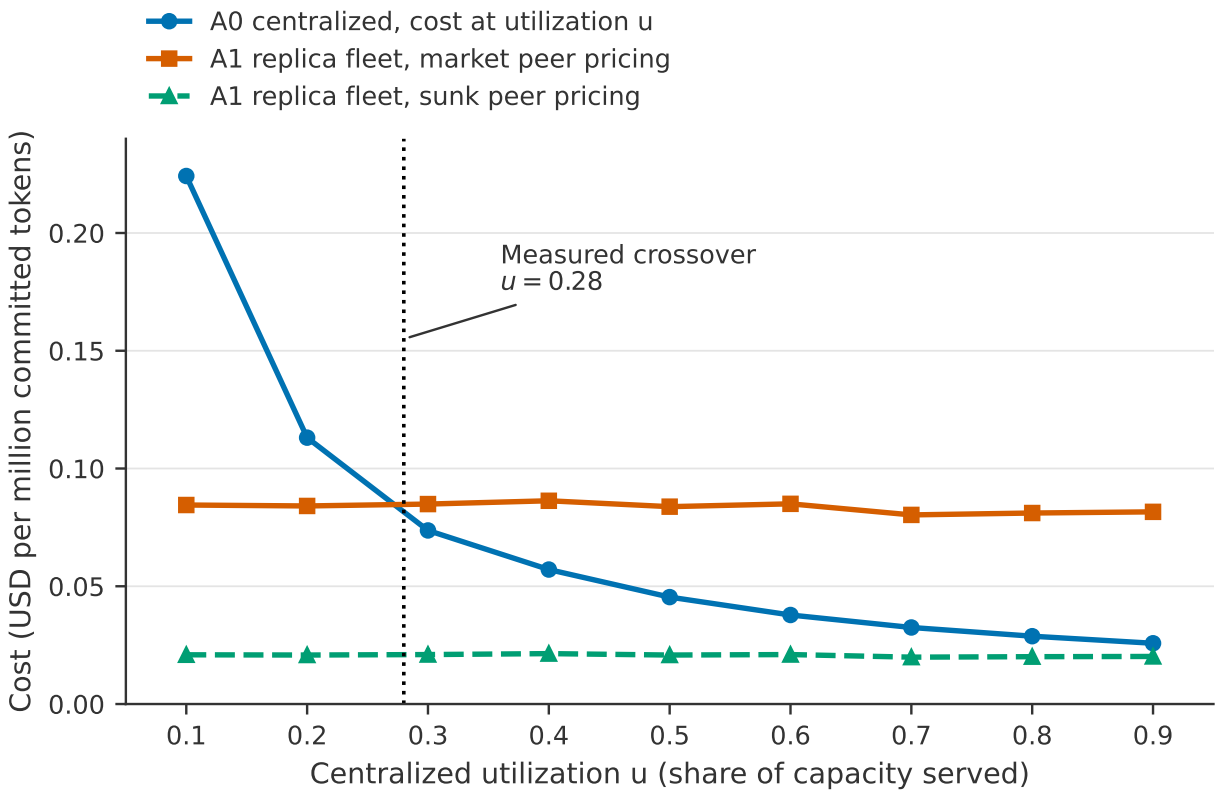


Figure 3. Cost per million committed tokens against centralized utilization u. A0 cost falls as $1/u$ (Eq. 10). A1 cost is flat in u under both peer pricing regimes because the replica fleet is provisioned to the offered load at headroom $\rho_{\max} = 0.7$. The market-regime curves cross at $u = 0.28$. Source: simulator, batch runs (MODELLED, C).

contest with A0 is purely economic: A1 wins on cost when the centralized alternative would run below $u \approx 0.28$ at market peer rates, or at any utilization when peer capacity is already owned and idle. The condition a deployment controls is the price ratio between data-centre hours and peer hours, not any property of the network.

Speculative decoding over a sharded verifier (A3) wins on mean throughput when the acceptance rate clears $\alpha^*(\text{RTT})$, 0.42 at city round trips and 0.17 at 150 ms, and it wins on system capacity when the verifier is contended, near 140 concurrent streams per verifier in the canonical case. It does not win on P95 gap at any acceptance rate, because the gap is set by the cycle time. A deployment that sells streaming smoothness should not buy A3 for that purpose; a deployment that sells batch throughput on a contended verifier can.

The restructured sparse design (A4R) closes exactly where the depth budget of Table 5 says it can: at the city tier, at $D \leq 4$ for a dGPU requester, and nowhere at continental or intercontinental round trips under a 50 ms target. Its break-even variables are the round-trip time of the routing neighbourhood and the funded cache size, which sets φ. Doubling the cache from 16 to 32 experts at $E = 256$ would raise φ from 0.55 to about 0.66 (RECONSTRUCTED, C) and buy depth, at 1.6 GB of device storage; the fleet-scaling test shows the same funding question returning at scale, because a fixed cache loses coverage as the model grows.

Sharding (A2) and per-layer expert routing (A4N) have no break-even region in the tested space.

Table 8. Hedged against unhedged A4R steps ($F = 500$, $r = 3$, continental).

	Bottom-decile share		P95 / P99 gap (ms)	
D	Unhedged	Hedged	Unhedged	Hedged
2	0.109	0.097	118 / 144	115 / 138
4	0.233	0.195	196 / 233	203 / 233

Notes: Bottom-decile share is the fraction of critical paths whose slowest step ran on a device in the bottom speed decile. Device speeds lognormal, $\sigma = 0.5$. Source: simulator, straggler runs (MODELLED, C).

Table 9. Use-case classification by design.

Design	Chat (P95 gap $\leq$ 50 ms)	Easy-query	Batch cost vs. A0 ref.
A0 centralized	Pass, all tiers	Pass, all tiers	Reference (0.045)
A1 replica routing	Pass, all tiers	Pass, city only	0.084, fail
A2 sharding	Fail, all tiers	Fail, all tiers	4.814, fail
A3 speculative ($k = 8$)	Fail, all tiers	Fail, all tiers	1.505, fail
A4R, $D = 2$	Pass, city only	Fail, all tiers	0.323, fail
A4R, $D = 4$	Pass, city only	Fail, all tiers	0.548, fail
A4N per-layer	Fail, all tiers	Fail, all tiers	112.184, fail

Notes: Batch cost in USD per million committed tokens at market peer pricing; the A0 reference is its cost at $u = 0.5$. Easy-query uses 32-token generations; A4R runs with early exit at $D = 2$ and A3 at $k = 4$ for that class. Source: simulator, use-case runs (MODELLED, C).

A2's deficit is structural, one traversal per stage added to an irreducible compute sum, and A4N's is the product of 58 sequential round trips with a per-token payload 1,200 times the dense boundary crossing.

6.2 Two gaps in the literature

The literature search behind Section 2 found no measured study of hedged requests for expert routing. Hedging is standard practice in data-centre serving and is analysed in the MoE placement literature, where providers replicate hot experts (DeepSeek-AI, 2026b), but the survey pool contains no measurement of what a hedged expert step does to tail latency. The measurement here, in simulation, is a null: at the tested operating point hedging moves work off slow devices without moving the P95 gap, because the residual tail is network jitter rather than device speed. That result is worth having in print precisely because the intuition behind hedging is strong and the conditions under which it pays, wider device-speed dispersion or longer step service times, are now specified by the failure.

The second gap is a combination. Locality routing, expert caching, and depth reduction each have published evidence behind them, from data-centre or edge testbeds (Section 2). No source in the pool evaluates the three jointly, on wide-area links, as one operating point with a single latency budget. A4R is that evaluation in model and simulation form. Its city-tier pass and continental-tier failure are properties of the combination, not of any one ingredient: caching without depth reduction leaves too many sequential steps, and depth reduction without caching leaves too high a remote fraction.

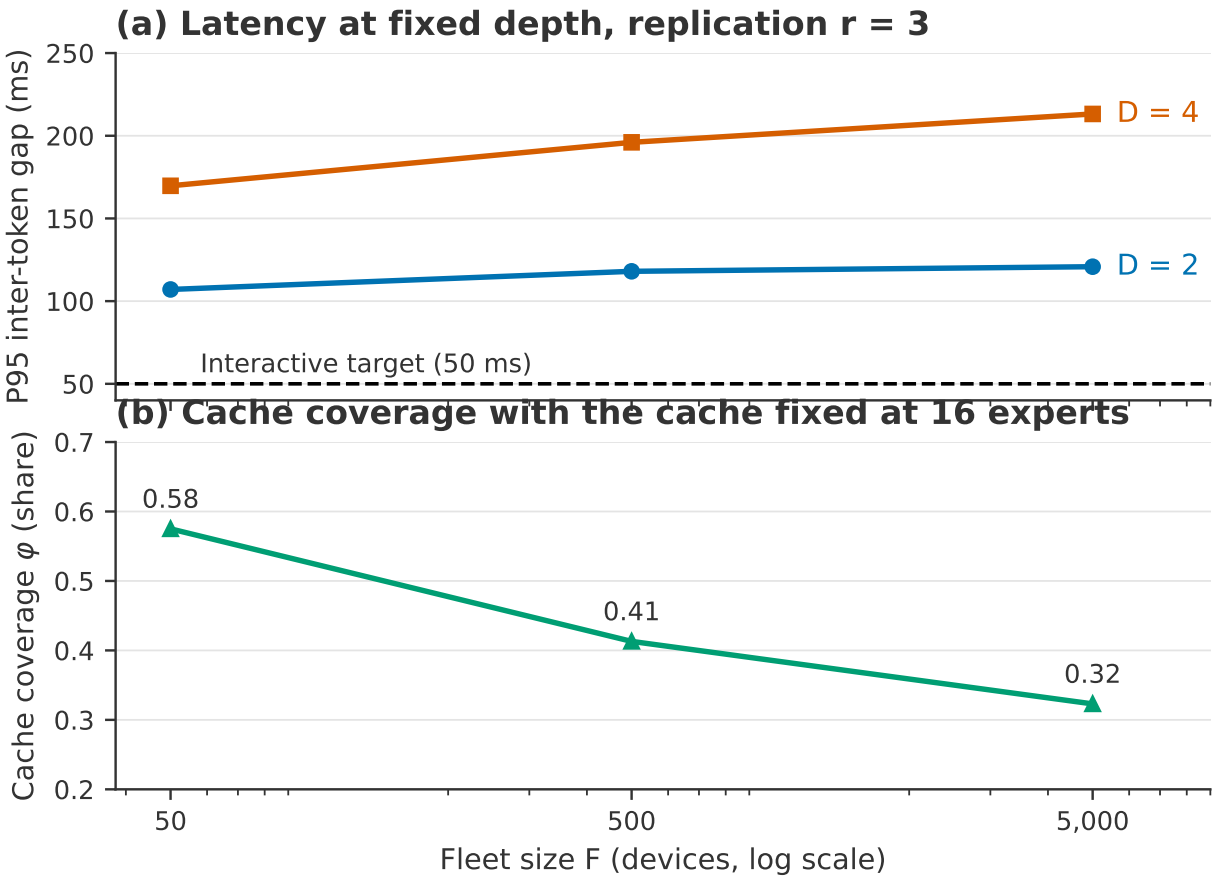


Figure 4. T-SCALE: fleet growth at fixed depth, continental tier, replication $r = 3$. Panel (a): P95 inter-token gap at $D = 2$ and $D = 4$. Panel (b): cache coverage φ with the per-device cache fixed at 16 experts, plotted as its own panel. The latency drift in panel (a) tracks the coverage decay in panel (b); no queueing term appears. Source: simulator, T-SCALE runs (MODELLED, C).

6.3 What a testbed phase would need to show

Every result in this paper is (MODELLED, C). A phase P2 testbed on physical devices would need to show five things to raise the grade of the conclusions.

First, it would need to reproduce the A2 and A4R P95 gaps on real links at the three tiers, with jitter and loss as measured rather than lognormal by assumption, and with NAT traversal in the path. The predictions to beat are Table 7, within a stated tolerance.

Second, it would need to measure an acceptance-rate distribution for a real draft and target pair on consumer hardware, locate the empirical α^* at two round-trip times, and compare them against the corrected Eq. (5) values of 0.42 and 0.17.

Third, it would need to measure φ under routing traces from a real workload rather than a Zipf-1 assumption, because the T-SCALE conclusion rests on how fast coverage decays as the expert count grows.

Fourth, it would need to measure peer departure in the middle of a generation. The simulator's churn treatment, in which an operation in flight completes and the failure is observed at the next

operation, is the most optimistic part of the model, and a testbed can replace it with a measurement.

Fifth, it would need to price verification: run TOPLOC spot checks at a stated sampling rate and report the added latency and device time, so the cost identity of Eq. (9) can be charged with a measured verification term instead of omitting it.

7 Limitations

All quantitative results in this paper are (Modelled, C). They are computed from the parameter values of Tables 1 and 2, which are (Assumed, C) for hardware and prices, and no hardware was measured. A reader who prices devices or links differently will move the cost crossover of Section 5 and the depth budgets of Table 5; the latency rankings of the arms are insensitive to prices but sensitive to the round-trip medians, which are assumed values.

The simulator carries simplifications that bound what its numbers can mean. Packet loss is held at zero in the reported grid, so the results isolate round-trip terms and say nothing about loss recovery on real access links; the kernel implements retransmission, but no reported run exercises it. Hedged steps are evaluated at the latency level, from current node loads, rather than by racing two full slot-model operations, which can only flatter hedging; the hedging null of Section 5 survives a flattering treatment, but a positive hedging result under this treatment would not have been believed. In A4 batch runs, the expert invocations that run parallel to the critical path are charged device time but not latency, which flatters A4R's tail in the same direction. Batch results for A0 and A1 come from a fluid model of the slot equations while A2, A3, and A4 batch results are per-token discrete-event runs, so cross-arm batch comparisons mix two instruments that agree on the closed forms but are not the same code path. Prompt length is 256 tokens throughout, except the easy-query class at 32-token generations, and prefill for A2 is charged to stages in equal shares.

Three modelling choices deserve explicit note. Expert popularity is Zipf with exponent 1 by assumption; measured popularity in a deployed router could be flatter or sharper, and the T-SCALE conclusion depends on the decay of φ under that law. Churn is optimistic: an operation in flight on a departing peer completes. Verification cost is not charged anywhere in the cost identity; Section 6 lists the measurement a testbed would need to supply it. The anchors transferred from data-centre results, the local- activation gains and caching gains behind A4R's design, are (Observed, B) in their home settings and are assumptions when moved to wide-area links. Finally, H4 is a consistency check against two observed operating points, not a test; the paper's training claims should be read at that strength.

8 Conclusion

A peer-to-peer fleet of consumer devices can serve open-weight models, in a bounded region of the design space that this paper maps. Whole- request routing to full replicas meets an interactive latency target at every network tier tested and undercuts centralized cost when centralized utilization would fall below 0.28, or at any utilization when the devices are already owned. A sparse model dispersed across the fleet meets the same target only inside one city, only at low sequential depth, and only when locality routing, expert caching, and early exit are funded together. Batch work at delay-tolerant settings is the natural load for the rest of the fleet's capacity.

The designs that split one token's computation across wide-area links fail on arithmetic, not on engineering detail. Sharding adds a round trip per stage to a compute sum it cannot reduce. Per-layer expert routing multiplies that error by 58 layers and a megabyte-scale payload. Speculative decoding is the partial exception: it raises mean throughput over a sharded verifier, more as links get slower, while its tail latency stays at the verification cycle time.

As the fleet grows, the constraint that binds is memory, in the form of cache coverage, rather than coordination or queueing. A model that grows in proportion to the fleet outruns any fixed per-device cache, and the latency cost of that decay is computable in advance from Eq. (8). The break-even conditions of Section 6 state what a deployment would have to fund to stay inside the region where the fleet works: replicas before shards, depth budgets before model size, and cache in proportion to the popularity mass the model is asked to cover.

References

blokzdev. The expert shuffle. Technical blog post, GitHub, 2026. Retrieved 16 October 2026.

Alexander Borzunov, Dmitry Baranchuk, Tim Dettmers, Max Riabinin, Younes Belkada, Artem Chumachenko, Pavel Samygin, and Colin Raffel. Petals: Collaborative inference and fine-tuning of large models. *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (System Demonstrations)*, pages 558–568, 2023. doi: 10.48550/arXiv.2209.01188. arXiv:2209.01188.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024. doi: 10.48550/arXiv.2401.10774.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023. doi: 10.48550/arXiv.2302.01318.

Qian Chen, Xianhao Chen, and Kaibin Huang. Slimcaching: Edge caching of mixture-of-experts for distributed inference. *arXiv preprint arXiv:2507.06567*, 2025. doi: 10.48550/arXiv.2507.06567.

Cointelegraph. io.net reveals network hit by sybil attack, reports fake GPUs inflating device counts. *Cointelegraph*, 2024. Retrieved 15 October 2026.

Damai Dai, Chengqi Deng, Chenggang Zhao, R. X. Xu, Huazuo Gao, Deli Chen, Jiashi Li, Wangding Zeng, Xingkai Yu, Y. Wu, Zhenda Xie, Y. K. Li, Panpan Huang, Fuli Luo, Chong Ruan, Zhifang Sui, and Wenfeng Liang. DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8082–8116, 2024. doi: 10.18653/v1/2024.acl-long.70.

DeepSeek-AI. Insights into DeepSeek-V3: Scaling challenges and reflections on hardware for AI architectures. *arXiv preprint arXiv:2505.09343*, 2025a. doi: 10.48550/arXiv.2505.09343.

DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2025b. doi: 10.48550/arXiv.2412.19437.

DeepSeek-AI. DeepEP: An efficient expert-parallel communication library. GitHub repository, 2026a. Retrieved 16 October 2026.

DeepSeek-AI. EPLB: Expert parallelism load balancer. GitHub repository, 2026b. Retrieved 16 October 2026.

distributed-llama contributors. distributed-llama: Inference of LLMs in a cluster or over the internet. GitHub repository, 2026. Retrieved 15 October 2026.

Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc'Aurelio Ranzato, Arthur Szlam, and Jiajun Shen. DiLoCo: Distributed low-communication training of language models. *arXiv preprint arXiv:2311.08105*, 2023. doi: 10.48550/arXiv.2311.08105.

Zhixu Du, Shiyu Li, Yuhao Wu, Xiangyu Jiang, Jingwei Sun, Qilin Zheng, Yongkai Wu, Ang Li, Hai Li, and Yiran Chen. SiDA: Sparsity-inspired data-aware serving for efficient and scalable large mixture-of-experts models. In *Proceedings of Machine Learning and Systems*, volume 6, pages 224–238, 2024. doi: 10.48550/arXiv.2310.18859. arXiv:2310.18859.

Exo Labs. exo: Run your own AI cluster at home with everyday devices. GitHub repository, 2026. Retrieved 15 October 2026.

Sami Jaghouar, Jack Min Ong, and Johannes Hagemann. OpenDiLoCo: An open-source framework for globally distributed low-communication training. *arXiv preprint arXiv:2407.07852*, 2024. doi: 10.48550/arXiv.2407.07852.

Albert Q. Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, Gianna Lengyel, Guillaume Bour, Guillaume Lample, Lélio Renard Lavaud, Lucile Saulnier, Marie-Anne Lachaux, Pierre Stock, Sandeep Subramanian, Sophia Yang, Szymon Antoniak, Teven Le Scao, Théophile Gervet, Thibaut Lavril, Timothée Laçon, Thomas Soulier, and Wojciech Stokowiec. Mixtral of experts. *arXiv preprint arXiv:2401.04088*, 2024. doi: 10.48550/arXiv.2401.04088.

Jakub Krajewski, Jan Ludziejewski, Kamil Pioro, Michał Pótyk, Piotr Bałas, Piotr Chrabaszcz, Jonas Björck, Michał Żbik, Grzegorz Michalski, and Piotr Miłoś. Scaling laws for fine-grained mixture of experts. *arXiv preprint arXiv:2402.07871*, 2024. doi: 10.48550/arXiv.2402.07871.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 19274–19286, 2023.

Yan Li, Pengfei Zheng, Shuang Chen, Zewei Xu, Yuanhao Lai, Yunfei Du, and Zhengang Wang. Speculative MoE: Communication efficient parallel MoE inference with speculative token and expert pre-scheduling. *arXiv preprint arXiv:2503.04398*, 2025a. doi: 10.48550/arXiv.2503.04398.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024. doi: 10.48550/arXiv.2401.15077.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-3: Scaling up inference acceleration of large language models via training-time test. *arXiv preprint arXiv:2503.01840*, 2025b. doi: 10.48550/arXiv.2503.01840.

Zonghang Li, Tao Li, Wenjiao Feng, Rongxing Xiao, Jianshu She, Hong Huang, Mohsen Guizani, Hongfang Yu, Qirong Ho, Wei Xiang, and Xue Liu. Prima.cpp: Fast 30–70b LLM inference on heterogeneous and low-resource home clusters. In *International Conference on Learning Representations*, 2026. doi: 10.48550/arXiv.2504.08791. arXiv:2504.08791.

Zixuan Liu, Zhiyong Chen, Nan Xue, Shengkang Chen, Jiangchao Yao, Meixia Tao, and Wenjun Zhang. WISV: Wireless-informed semantic verification for distributed speculative decoding in device-edge LLM inference. *arXiv preprint arXiv:2604.17701*, 2026. doi: 10.48550/arXiv.2604.17701.

Messari. State of render, Q3 2025. Technical report, Messari, 2025. Retrieved 15 October 2026.

Messari. State of akash, Q4 2025. Technical report, Messari, 2026. Retrieved 15 October 2026.

mmprotest. swarm-inference-research: Phase 4 sealed three-node wide-area inference test. GitHub repository, 2026. Retrieved 16 October 2026.

Nous Research. DisTrO: Distributed training over the internet. Technical report, Nous Research, 2024. Project report, retrieved 15 October 2026.

Jack Min Ong, Matthew Di Ferrante, Aaron Pazdera, Ryan Garner, Sami Jaghouar, Manveer Basra, Max Ryabinin, and Johannes Hagemann. TOPLOC: A locality sensitive hashing scheme for trustless LLM inference proof verification. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, 2025. doi: 10.48550/arXiv.2501.16007. arXiv:2501.16007.

Prime Intellect. INTELLECT-1 technical report. Technical report, Prime Intellect, 2024. arXiv:2412.01152.

Prime Intellect. INTELLECT-2: Massively scaling reinforcement learning with internet compute. Technical report, Prime Intellect, 2025. arXiv:2505.07291.

David Raposo, Sam Ritter, Barry Richards, Edouard Grave, Alexander Pritzel, and Arthur Guez. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. In *arXiv preprint arXiv:2404.02258*, 2024. doi: 10.48550/arXiv.2404.02258.

Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. SWARM parallelism: Training large models can be surprisingly communication-efficient. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 29416–29440, 2023. doi: 10.48550/arXiv.2301.11913. arXiv:2301.11913.

Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Q. Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *arXiv preprint arXiv:2207.07061*, 2022. doi: 10.48550/arXiv.2207.07061.

Jingwei Song, Wanyi Chen, Xinyuan Song, Max, Chris Tong, Gufeng Chen, Tianyi Zhao, Eric Yang, Bill Shi, and Lynn Ai. Speculative decoding in decentralized LLM inference: Turning communication latency into computation throughput. *arXiv preprint arXiv:2511.11733*, 2025. doi: 10.48550/arXiv.2511.11733.

Lixiang Tong, Hui Jiang, Sishen Chen, Yukuan Zhao, Xiangxiang Lu, Wenhao Qu, Jiajun Yang, Sikai Ai, and Anmin Yuan. Parallax: Efficient LLM inference service over decentralized environment. *arXiv preprint arXiv:2509.26182*, 2025. doi: 10.48550/arXiv.2509.26182.

Table 10. Minimum device specification by role.

Role	Memory (GB)	Bandwidth (GB/s)
Replica (M8 INT4)	≥ 6.0	≥ 114
Shard (quarter of M70 INT4)	≥ 9.9	≥ 250; ≥ 125 relaxed
Draft only (D1 INT4)	≥ 1.2	≥ 60; ≥ 36 relaxed

Notes: Memory figures include weights, key-value cache, and runtime headroom. Bandwidth thresholds invert Eq. (1) at a step bar of 50 ms for the replica, 12 ms for the draft, and a 20 tokens per second chain rate for the shard; the relaxed column values correspond to a 20 ms draft step and a 10 tokens per second chain. Source: computed (Reconstructed, C) from (Assumed, C) hardware parameters.

Table 11. Role verdicts by device class.

Class	Replica	Shard	Draft	A4 expert host
IoT / TV SoC	No	No	Marginal	Yes
Smartphone SoC	No	Batch only	Yes	Yes
Laptop iGPU	No	Batch only	Yes	Yes
Consumer dGPU	Yes	Yes	Yes	Yes
Edge server	Yes	Yes	Yes	Yes

Notes: Smartphone replica fails on speed, not memory: its M8 INT4 step is 95 ms, 10.5 tokens per second, below the 20 bar. The laptop iGPU reaches 17.5 tokens per second, just below the bar. IoT draft is marginal at a 28.6 ms draft step. The consumer dGPU shard verdict assumes $N = 4$; an $N = 2$ shard of 17.5 GB does not fit its 12 GB. The edge server also holds a full M70 INT4 replica (step 25 ms). Source: computed (Reconstructed, C).

Fengze Yu, Leshu Li, Brad McDanel, and Sai Qian Zhang. DSD: A distributed speculative decoding solution for edge-cloud agile large model serving. *arXiv preprint arXiv:2511.21669*, 2025. doi: 10.48550/arXiv.2511.21669.

A Device-class thresholds

The model implies a minimum specification for each role a device can play, from Eq. (1) and the memory arithmetic of Section 3. Table 10 states the thresholds. Table 11 applies them to the device classes of Table 2. The structural reading is that phones and IoT devices qualify as draft nodes, cache nodes, and expert hosts, and do not qualify as replica nodes at the interactive bar of 20 tokens per second.

B Full results

The complete machine-readable record of the simulation campaign is the file `matrix.csv` in the study's results directory (113 data rows), regenerated exactly by rerunning the run matrix with the recorded seeds. Tables 12 and 13 print the two result families that the main text summarizes.

Table 12. Batch mode: aggregate throughput and cost per committed token.

Configuration	Aggregate (tok/s)	USD/Mtok, market	USD/Mtok, sunk
A0 at $u = 0.5$ (fluid)	12,475.5	0.045	0.045
A1 fleet, matched load (fluid)	12,547.3	0.084	0.021
A2 chain, 16 streams	68.3	4.814	2.158
A3 sharded verifier, 16 streams	202.7	1.505	0.688
A4R, $D = 2$, 32 streams	770.7	0.323	0.251
A4R, $D = 4$, 32 streams	393.1	0.548	0.476
A4N, 4 streams	0.5	112.184	111.572

Notes: Market and sunk regimes price peer device time at the two rates of Table 2. A0 and A1 batch rows are fluid-model results; the remaining rows are per-token discrete-event results (Section 7). Source: simulator, batch runs (Modelled, C).

Table 13. T-SCALE interactive results, full quantiles ($r = 3$, continental).

		$D = 2$ gap (ms)			$D = 4$ gap (ms)		
F	φ	P50	P95	P99	P50	P95	P99
50	0.58	41	107	130	76	170	209
500	0.41	51	118	144	111	196	233
5,000	0.32	65	121	141	130	213	242

Notes: Unique experts $E_u = 4F$; total parameters 20B, 200B, and 2T at $F = 50$, 500, and 5,000. Per-device storage is constant: eight expert copies (400 MB) plus the fixed 16-expert hot cache. Eight active experts per token, Zipf exponent 1. Source: simulator, T-SCALE runs (Modelled, C).

C Falsifiability

Each main conclusion is listed with the evidence that would overturn it. The hypotheses H1 to H4 were fixed in the research design before the simulator was run; beyond that design document, no pre-registration exists, and this appendix is a retrospective statement of what would change the paper's conclusions.

C1. Sharding never beats a smaller local model in single-stream latency. Overturned by one configuration on homogeneous hardware at a wide-area round trip of 30 ms or more in which a sharded model's measured P95 inter-token gap falls below the local smaller model's, at matched output quality.

C2. The speculation break-even falls as round-trip time rises, and the P95 gap of speculative serving sits at the cycle time. Overturned by a measured break-even acceptance rate that rises with round-trip time, or moves by more than 0.30 between the city and 150 ms tiers under the corrected Eq. (5), or by a speculative deployment over a sharded verifier whose P95 gap tracks its mean gap rather than its cycle time.

C3. Peer replica serving undercuts centralized cost only below a utilization crossover in the region of 0.28 to 0.35. Overturned by batch cost measurements at the stated market rates showing no crossing, or A1 cheaper at $u = 0.9$, or A0 cheaper at $u = 0.1$.

C4. Per-layer expert routing (A4N) is infeasible on wide-area links. Overturned by a wide-area deployment of per-layer expert routing sustaining an interactive P95 gap on consumer

links, which would require the effective per-token communication to fall far below the volume and message-count bounds of Section 3.5.

C5. The restructured sparse design (A4R) meets a 50 ms interactive target only at city round trips and low depth. Overturned by a measured A4R-class operating point at continental round trips and $D \geq 2$ holding a P95 gap at or below 50 ms, or by a city-tier failure at $D \leq 4$ under the model's parameters.

C6. At fixed depth, fleet growth moves latency through cache coverage, not queueing. Overturned by a scaling run in which latency degrades with fleet size while cache coverage is held constant by funding the cache in proportion to the expert count, or in which a queueing term appears at aggregate rates within the tested range.

C7. Hedging expert steps does not reduce tail latency at the tested operating point. Overturned by a measured P95 reduction from hedging at matched cost in a rerun at the same point, or by evidence that the null is an artefact of the simulator's latency-level treatment of hedged steps (Section 7); the claim is stated for one operating point and does not extend to fleets with wider device-speed dispersion.

C8 (H4). Internet training is feasible only in the low-communication regime. Overturned by a demonstrated synchronous data-parallel run of an 8B-class model over uplinks of at most 100 Mb/s with synchronization at or below 10 per cent of step time, or by a low-communication run failing at an inner-step count at or above the bound of Eq. (11).

Production note. This paper was produced by Kilambi Research's quantitative research pipeline with AI assistance; all analysis was verified against source data.